

Annex XVII — WR Entries™ and the Publisher Console

Value-Composed Public Entry, Immediate Value, Public Auditability, and the Authoring Plane

BEAP™ / WR Desk™ / Optirando™ Canonical Specification — standalone Annex. Version 1.0 — 7 August 2026 · Part of the Optirando™ Canon · Status: Defensive Publication / Prior-Art Disclosure

Annex number provisional — assign per Canon sequence.

Document status: Normative unless explicitly marked as informative.

Applies to: publisher-defined WR Entries as value-composed custom public handshakes; the Entry Value Package as the entry-class unit of immediate, verified value; value composition by reference, template semantics, and preparation tooling; universal public auditability of publisher material; the Publisher Console as the zero-authority authoring surface over the orchestrator administration model; and modular MVP staging of the authoring plane.

Trademark notice: Optirando™, WR Desk™, WR Graph™, BEAP™, PoAE™, PoAC™, WRVault™, WR Guard™, WR Entry™, Fraud Watchdog™, Workflow Ready Cloud (WRC)™, and WR Code® are used throughout as product and protocol designations of the Optirando™ system.

XVII.0 Status, Scope, and Relationship to the Canon

This Annex specifies the publisher-side **authoring plane**: how a publisher composes, offers, and maintains individually valuable public entry points, and how anyone audits what was published. It is deliberately implementation-guiding: §XVII.3–§XVII.5 are the data contracts, §XVII.6 the audit contract, §XVII.7 the interaction contract, §XVII.8 the build order.

This Annex does not redefine wire formats, cryptographic primitives, handshake formation, scope semantics, loading, designation, or execution. It depends on, reuses, and does not weaken:

- **Annex VII** — the artifact and manifest machinery, publisher attestation, and the grant model (delivery and preparation rights), with which the Execution Authorization Declarations referenced here compose;
- **Annex IX** — the initiation layer: entry exclusively per §IX.3.1 (the code is an identifier and invitation, never authority), capture methods, dual-channel publisher verification (§IX.3.5), Hash-Pinned Consent, and the invitation discipline;
- **Annex XIV** — Scoped Worlds, and specifically §XIV.5.5 (Execution Authorization Proof Chain and Catalog Commitment): the Catalog Head, the proof chain, delegation, dual assurance, and the entry-class/selection-class load discipline. Every object this Annex defines is a catalog object under that machinery;

- **Annex XI** — the WR Pointer as designation layer; pointer capabilities appear here only as composable, referenced value;
- **Annex III** — PoAC™ for every capability admission;
- **Annex X** — WR Guard™ on every external path.

The core philosophy of the Canon applies without exception: the system prepares, the human authorizes, the system records cryptographic proof. Nothing in this Annex creates, implies, or pre-satisfies any execution authority; the authoring plane produces declarations, never permissions-in-effect.

XVII.0.1 Claims Discipline (Normative)

The claims discipline of Annex IX §IX.0.1 applies verbatim. Security statements herein are either properties by construction or objectives pending validation (§XVII.9). Nothing herein SHALL be represented as a certified or formally verified system.

XVII.1 Motivation and Problem Statement (Informative)

Annex XIV solves what an entry *opens* — a governed scope, loaded on selection. This Annex solves what an entry *offers*, and to whom the offering must be legible at three moments:

1. **To the visitor, immediately.** A security architecture whose first render is bureaucracy loses the user before the first selection. The value of an entry must be conveyed immediately — and "immediately" must be an engineered guarantee, not a hope, and must never be bought by weakening verification.
2. **To the publisher, simply.** A machine manufacturer, a shop, an event organizer, and a one-person website have entirely different offerings but must not need different mental models — and none of them may be forced to understand the whole Canon to publish one good entry.
3. **To everyone, verifiably.** Publisher material that claims to be governed must be checkable by anyone, from a link, without an account — otherwise "verified" is a UI claim, not a property.

The resolution: the **WR Entry™** as a named, value-composed view onto the one proof chain; the **Entry Value Package** as the size-budgeted first verified render; the **Publisher Console** as a zero-authority UI that compiles picking-and-arranging into the same signed objects the orchestrator has always administered; and **public auditability** as a stable, unauthenticated repository URL for every published thing.

XVII.2 Definitions

Terms defined in Annexes II–XV apply as defined there. This Annex adds:

WR Entry™. A publisher-defined, individually addressable public handshake offering: a named entry point bound to a selection of the publisher's scopes and to an Entry Value Package. A "custom public handshake" in publisher language; cryptographically, a catalog object under the Catalog Head (Annex XIV §XIV.5.5).

Entry Value Package (EVP). The entry-class content unit (Annex XIV §XIV.5.5(6)) of a WR Entry™ for non-spatial entry: the signed self-description, value statement, scope directory, optional preparation view, and audit links — under a hard size budget.

Value Statement. The short, human-readable, publisher-signed statement of what an entry offers. Part of the EVP and thereby part of the proof chain.

Scope Directory. The EVP's map of the offering: scope names, icons, one-line descriptions, size and prefetch hints. Never artifacts, never member content.

Template. A reusable entry composition (default scope structures, default capability assignments, default EVP layout) instantiated by copy-on-assign (§XVII.5.2).

Preparation Tool. Platform-provided support tooling whose output becomes ordinary publisher material (§XVII.5.3).

Publisher Console. The administration surface for WR Entries: a user interface over the orchestrator administration model of Annex XIV §XIV.5.5(4), holding no keys and no authority (§XVII.7).

Organigram View / Console Catalog. Terms of the informative reference interaction model (§XVII.7.2): the hierarchical chart of the publisher's entries and the extensible catalog of everything that can be offered. The Console Catalog is a UI surface and is distinct from the publisher's cryptographic catalog under the Catalog Head.

XVII.3 The WR Entry™

XVII.3.1 One Trust Mechanics, Many Compositions (Normative)

Entries differ in VALUE COMPOSITION only, never in trust mechanics. Every entry resolves through the identical proof chain of Annex XIV §XIV.5.5; no entry type, template, or archetype carries more or less authority than another, and no entry weakens any rule of Annex IX entry (capture per §IX.3.1; surfaces zero-authority per §XIV.6.1).

XVII.3.2 The Entry Object (Normative)

Normative fields:

- `entry_id` — stable identifier within the publisher namespace;
- `display` — name, icon reference, and the value statement (rendered per §XVII.4);
- `codes` — the WR Code(s) and capture channels this entry answers to: code scan, manual entry, assisted email, assisted discovery, and (where Annex XV applies) entrance scan;
- `scopes` — the grouped contexts this entry exposes (references into the publisher catalog; one or more);
- `evp_ref` — content hash of the entry's Entry Value Package;
- `template_ref` — provenance of the template this entry was instantiated from, if any (§XVII.5.2; provenance only, no runtime authority);
- `status` — draft | published | suspended | retired;
- `epoch` and publisher signature (root or delegated Catalog Signing Key).

Entries are catalog objects: listed under the Catalog Head like Scope Manifests, covered by the same commitment, inclusion proofs, epochs, anti-rollback, and freshness, and administered exclusively through the Publisher Orchestrator.

XVII.3.3 Entry Lifecycle (Normative)

draft is Console-local and unsigned-published (it MAY be orchestrator-signed as draft for team workflows but is never resolvable by clients). **published** requires: valid EVP within budget, all composition references resolvable under the current Catalog Head, and dual assurance present on every referenced artifact. **suspended** withdraws resolvability without deleting history; **retired** is terminal. Status transitions are epoch events under the catalog commitment.

Publication is attested handover (Normative). Authoring and signing exist exclusively at the Publisher Orchestrator; the platform account offers NO draft creation or editing and is read-only toward the catalog (published state, audit links, ingest results). Publishing is the attested handover of a **publication package** to the **Workflow Ready Cloud (WRC)**TM repository: the new or changed signed objects (entries, EVPs, Scope Manifests, Execution Authorization Declarations — content-addressed, signed under the Catalog Signing Key) together with the new Catalog Head at epoch $n+1$, transferred over the attested publisher channel that alone authorizes writes to the publisher namespace. Ingest applies the repository discipline of Annex XIV §XIV.5.5(5) and, on success, attaches the ingest countersignature; only then do the objects become resolvable and their audit URLs live. Two rules bind the handover:

1. **Atomicity per epoch.** A publication becomes resolvable as a whole — the complete new Catalog Head with every referenced object — or not at all. No partial catalog state ever exists; no entry may become resolvable while referencing an object that is not.
2. **Rejection, never modification.** The platform MAY reject a publication (budget violation, scan finding, unresolvable reference) with a stated reason returned to the orchestrator; it NEVER alters, amends, re-signs, or partially applies one. The ingest countersignature is a hygiene attestation, never an editorial act. A rejected publication leaves the previous epoch in force.

Platform-side scanning and suspension (Normative). **Fraud Watchdog**TM is a family of fine-tuned models and adapters (e.g. LoRA) deployed not only locally on-premises — on the orchestrator, in its publisher-side role — but also in the **Workflow Ready Cloud (WRC)**TM: as part of the WRCTM infrastructure it scans published public content on an ongoing basis to detect and flag suspicious material. Upon a finding, the platform MAY **suspend** the resolvability of the affected public content. Suspension is the platform-side counterpart of the publisher's **suspended** status and stands under the same discipline as ingest: it withdraws resolvability visibly, with a stated reason to the publisher and a visible marker in the public audit view — it never edits, re-signs, or replaces content (suspension, never modification). Suspension is reversible upon resolution, and the record of the suspension remains auditable. **Fraud Watchdog**TM is one layer of the security stack, never a substitute for the proof chain, for WR GuardTM path governance, or for operator-side consent.

XVII.4 The Entry Value Package — Immediate Value, Made Explicit

XVII.4.1 The Immediacy Principle (Normative)

The value of an entry **MUST** be conveyed immediately, as far as possible — and "as far as possible" is bounded by verification, never the other way around. Verify-first is never relaxed for perceived speed: no unverified content is ever rendered. Immediacy is achieved architecturally, by making the first verifiable unit small.

XVII.4.2 EVP Contents (Normative)

All covered by the publisher signature:

1. the publisher self-description and the entry's value statement;
2. the scope directory (§XVII.2) — the map of the offering, never its content;
3. optionally ONE preparation view of the most probable initial context, subject to the preparation rules (marked inferences, visible provenance);
4. next-step affordances: what selection will unlock — declared, not preloaded;
5. the audit link set for the entry (§XVII.6).

XVII.4.3 The Size Budget (Normative)

The EVP carries a declared maximum size, fixed platform-wide per client class, chosen so that transfer plus full verification completes below the perception threshold on the weakest supported client class. Immediacy is thereby a guaranteed property of the class. An EVP over budget is **rejected at ingest, never truncated at render**.

XVII.4.4 Anti-Spoofing of the Value Promise (Normative)

After capture, the client renders **ONLY** the signed value statement and self-description from the verified EVP — never the carrier's claim. A badge, website, poster, or email may assert any value it likes; the client's first render shows what the publisher cryptographically committed to, and any discrepancy is thereby visible to the operator. The value promise is part of the proof chain. (This composes with the Annex IX invitation discipline: carriers show identifiers and invitations; the verified claim lives behind verification.)

The first verified render of an entry is thus: verified identity, verified value statement, verified map of the offering, and a clear next step — with nothing bulk-loaded. Selection then proceeds per Annex XIV §XIV.5.

XVII.5 Value Composition, Templates, and Preparation Tools

XVII.5.1 Composition by Reference (Normative)

A publisher assigns value to an entry by reference: scopes, templates, adapters (e.g. safetensors), configurations, pointer AI automation capabilities, preparation views. Every assignment references a hash-identified catalog object carrying its Execution Authorization Declaration per Annex XIV

§XIV.5.5(3). Assignment never copies content into the entry; an entry remains a composition of references, and removing a reference is a catalog change under the ordinary epoch discipline.

XVII.5.2 Templates and Copy-on-Assign (Normative)

1. **Standard entry types ship with default templates** — e.g. a *Website entry* template whose default composition includes the WR Pointer with navigation assistance; further standard types (shop entry, event entry, machine entry) follow the same pattern. Defaults are fully editable by the publisher.
2. **Copy-on-assign:** instantiating a template copies its composition into the entry at that moment. Later changes to the template NEVER silently mutate existing entries; updating an entry to a newer template version is an explicit, publisher-confirmed re-apply. Silent template propagation would be an unauthorized change to a signed, published composition and is non-conforming.
3. `template_ref` records provenance for Console and audit; it carries no runtime authority.

XVII.5.3 Preparation Tools (Normative)

The platform MAY provide support tooling that helps publishers produce material — for example a website scanner that prepares AI navigation assistance for a site, and successive further tools of this kind. Rule: **a tool's output is ordinary publisher material**. It enters the catalog through the same ingest, receives the same dual assurance (publisher signature = authorization; ingest countersignature = hygiene), stands under the same commitment, and is equally auditable. Tools accelerate authoring; they NEVER bypass, shortcut, or substitute any link of the proof chain. Tool provenance is recorded in the material's metadata.

XVII.6 Public Auditability

Every publisher material MUST be verifiable and auditable by anyone, without any handshake, account, or client:

1. every catalog object — entry, EVP, Scope Manifest, artifact — has a **stable public repository URL** under the publisher's namespace, resolvable read-only and unauthenticated;
 2. the repository view of an object presents: the content hash, the publisher signature, the platform ingest countersignature, the catalog epoch, and the inclusion proof against the current Catalog Head;
 3. the client offers a **verify-in-repository link** for every rendered item, including inside the EVP's audit link set — from anything the client shows, a user can jump to its public, independently checkable record;
 4. content addressing yields the transparency property: what the client loaded and what the repository serves under that hash are byte-identical — or the mismatch is itself the finding;
 5. audit access is passive: the public view never executes, personalizes, or tracks; it displays and proves. (Information without authority, Annex II §II.2.5.)
-

XVII.7 The Publisher Console

XVII.7.0 Zero Authority (Normative)

The Console is a user interface over the administration model of Annex XIV §XIV.5.5(4): everything it does compiles into the same signed objects (entries, EVPs, Scope Manifests, declarations, Catalog Heads), signed via the Publisher Orchestrator. The Console holds no keys and introduces NO additional authority; a Console defect can produce an unpublishable draft, never an unauthorized publication.

XVII.7.1 Console Invariants (Normative)

Whatever interaction design an implementation chooses, the following invariants bind it:

1. **Presentation carries no semantics.** Visual organization — grouping, arrangement, ordering, theming — carries no trust or runtime semantics; nothing about how entries are displayed changes what is signed.
2. **No phantom structure.** The Console never displays a structure, state, or relationship that does not exist cryptographically in the signed objects.
3. **Uniform authority across presets.** No archetype, template pack, starting point, or preset differs in security model, verification, or authority; differentiation across publisher kinds lives entirely in the value-composition layer.
4. **Truthful state.** An entry's real published state (epoch, current Catalog Head, audit link) is always distinguishable from its draft state; the two are never conflated, and no "published" indicator may be rendered that the repository cannot corroborate.
5. **A minimal path exists.** Publishing one entry from a standard type with its defaults MUST be possible without touching any advanced surface and without understanding the system beyond that one entry.
6. **Single authoring instance.** Authoring and signing exist exclusively at the Publisher Orchestrator (§XVII.3.3); the platform account creates no drafts, edits nothing, and is read-only toward the catalog. A compromised platform account can therefore neither publish nor change anything — it can only read what is publicly auditable anyway.

XVII.7.2 Reference Interaction Model (Informative)

The following describes ONE interaction design satisfying §XVII.7.0–§XVII.7.1 — the current working concept for the Console at the Publisher Orchestrator. The Optirando™ account surface is a read-only view of published state, audit links, and ingest results per invariant 6 and carries no authoring UI. It is guidance, not contract: implementations MAY depart from it freely and without amendment as long as the invariants hold; UI iteration is expected and intended.

- A hierarchical **organigram view** as the primary surface: the publisher's offering rendered as an expandable chart — groups as branches (visual only, per invariant 1), WR Entry™ objects as nodes, and each entry's assigned items (scopes, capabilities, materials) as child nodes beneath it. Expanding and collapsing follows the signed hierarchy exactly (invariant 2); the chart pans, zooms, and scrolls without pagination.
- A second surface, the **Console Catalog**: the extensible, searchable overview of everything that can be offered — standard entry types, templates, capability classes, preparation tools — with plain-language descriptions. Composing is picking from the Console Catalog onto

an entry; new platform capabilities appear as new catalog items without changing the Console shell.

- **Progressive disclosure:** a collapsed node shows only name, value statement, status, and a count of assigned items; everything else sits one expansion level deeper, and advanced options never appear on the first screen.
 - **Archetype starting points** — machine manufacturer, website operator, shop, event provider, and further archetypes over time — as preselected templates and defaults only (invariant 3).
-

XVII.8 Modularity and MVP Staging

The Console and entry system **MUST** be built modularly such that a minimal version is complete-but-small rather than broad-but-weak (Normative).

Reference staging (Informative — the module cut follows the reference interaction model of §XVII.7.2 and may be re-cut with it):

- **M0 (MVP):** one standard entry type (Website entry) with its default template (WR Pointer with navigation assistance); the organigram view without grouping (flat root level); the Console Catalog read-only with the shipped defaults; EVP authoring limited to value statement plus auto-generated scope directory — and the FULL proof chain, dual assurance, and public auditability of Annex XIV §XIV.5.5 and §XVII.6.
- **M1:** grouping; additional standard entry types; template editing; custom compositions.
- **M2:** preparation tools (website scanner and successors); archetype packs; predictive preparation hints; EVP preparation views.

Staging rule, absolute (Normative): staging only ever reduces convenience, never assurance. Verification, the proof chain, dual assurance, and public auditability are first-stage properties and are never optional, deferred, or feature-flagged. A stage shipped without them is not an earlier version of this system; it is a different and non-conforming system.

XVII.9 Security Model

XVII.9.1 Properties by Construction (Normative Claims)

- No entry type, template, or archetype alters trust mechanics; every entry resolves through the unchanged proof chain (§XVII.3.1).
- No unverified content is ever rendered; immediacy derives from the EVP size budget, never from relaxed verification (§XVII.4.1, §XVII.4.3).
- The first render after capture shows the publisher's signed value claim, never the carrier's — a lying carrier is visible by comparison (§XVII.4.4).
- Template changes never silently mutate published entries (§XVII.5.2).
- Tool output holds no privileged path into the catalog (§XVII.5.3).

- Every published object is publicly resolvable with hash, signatures, epoch, and inclusion proof; client-loaded and repository-served content are byte-identical under content addressing (§XVII.6).
- The Console can draft but never publish outside the orchestrator signing path; a Console compromise cannot mint authority (§XVII.7.0).
- Authoring is single-instance: the platform account can neither draft, publish, nor modify; account compromise yields read access to publicly auditable data only (§XVII.7.1(6), §XVII.3.3).
- Publication is atomic per epoch, and rejection never modifies: no partial catalog state exists, and no platform-side alteration of a publication is possible (§XVII.3.3).
- Platform-side suspension withdraws resolvability visibly — stated reason, auditable record — and never modifies content (§XVII.3.3).
- No stage of the system exists without verification, dual assurance, and auditability (§XVII.8).

XVII.9.2 Objectives Pending Validation (Informative)

Choice and calibration of the EVP size budget across client classes; UX efficacy of the value-statement anti-spoofing comparison for non-expert users; scalability of the public audit view for very large catalogs; Console usability across archetypes; draft-workflow security in multi-editor publisher teams.

XVII.9.3 Non-Goals (Normative)

This Annex does not guarantee that a value statement is commercially truthful (it proves who committed to it, not that it is wise); does not define repository storage or CDN architecture; and does not replace the entry, scope, or execution rules of Annexes IX, XIV, and XI, which apply unchanged.

XVII.10 Open Specification Items (Informative)

EVP serialization and the per-client-class size budget values; scope-directory entry format (icons, hints); entry object serialization and the entry/EVP hash-tree placement under the Catalog Head; audit-view HTML/JSON representation and its inclusion-proof presentation; Console draft-object format and multi-editor semantics; template package format and re-apply diff presentation; preparation-tool provenance metadata schema; archetype pack format.

XVII.11 Extension Points and Amendment Discipline

Declared extension points (additive only): new standard entry types and templates; new archetypes; new Console Catalog item classes; new preparation-tool classes; additional EVP content classes within the size budget. Amendments follow the Canon discipline: versioned, never silent, recorded in §XVII.12; changes to normative invariants require explicit identification and justification (Annex XI §XI.0 pattern).

XVII.12 Version History

Version	Date	Change
1.0	2026-08-07	Initial normative specification of the authoring plane: the WR Entry™ as value-composed catalog object with identical trust mechanics across all entries and an epoch-governed lifecycle with publication as attested handover (single authoring instance at the orchestrator, read-only platform account, per-epoch atomicity, rejection-never-modification, and ongoing platform-side Fraud Watchdog™ scanning with visible, auditable, reversible suspension — never modification); the Entry Value Package with immediacy principle (bounded by verification), size budget with ingest rejection, and anti-spoofing of the value promise as part of the proof chain; composition by reference with copy-on-assign template semantics, default templates per standard entry type, and preparation tools whose output is ordinary dual-assured publisher material; universal public auditability (stable unauthenticated URLs, hash + signature + countersignature + epoch + inclusion proof, verify-in-repository links, passive audit view); the Publisher Console as zero-authority UI bound by normative invariants (presentation carries no semantics, no phantom structure, uniform authority across presets, truthful state, guaranteed minimal path), with the organigram/Console-Catalog interaction design carried as an informative reference model; modular staging with an informative M0/M1/M2 reference cut under the absolute normative rule that staging reduces convenience, never assurance. Depends on Annex XIV §XIV.5.5 (proof chain and catalog commitment).

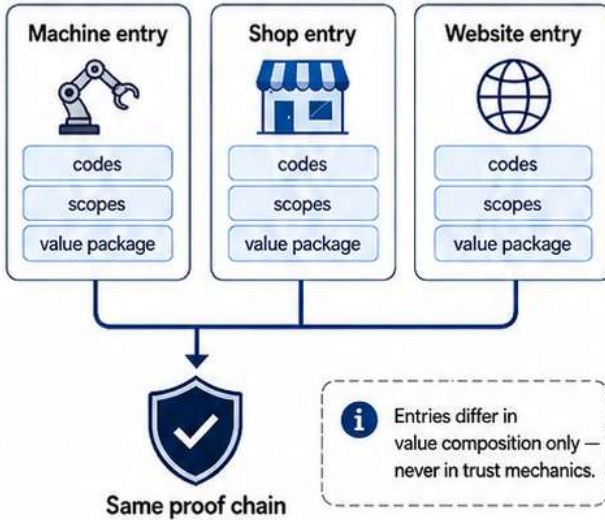
License Notice

This Annex forms an integral part of the Optirando™ specification and is licensed under the same terms as BEAP™, WR Desk™, and PoAE™.

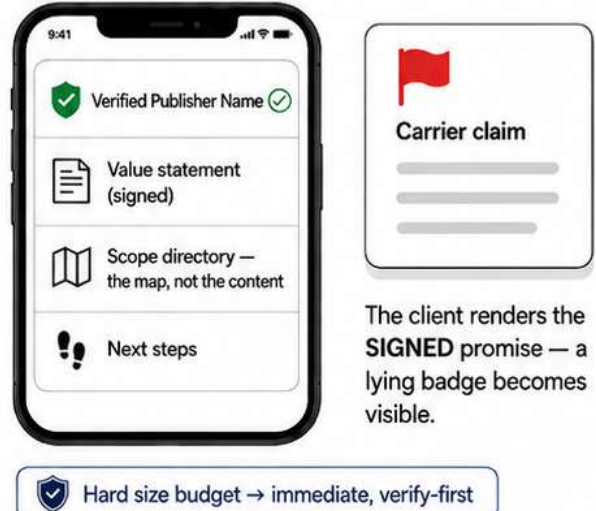
WR Entries™ — Custom Public Handshakes, One Trust Mechanics

Publishers compose the value. Verification never changes. Everyone can audit.

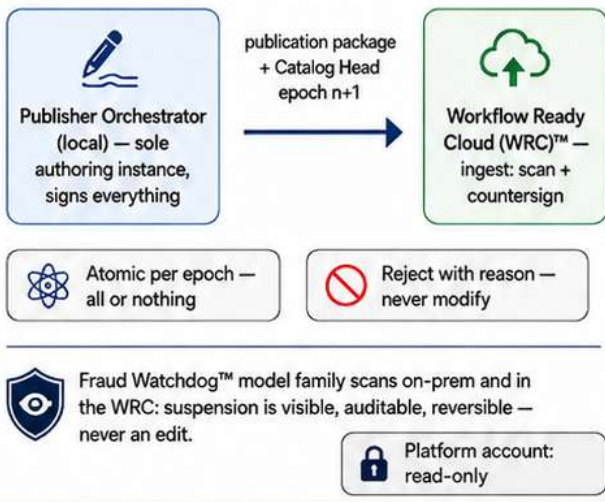
1 One entry, many compositions



2 First verified render: the Entry Value Package



3 Publishing = attested handover



4 Anyone can audit



What This Guarantees

- ✓ verified value before belief
- ✓ single signing authority at the publisher
- ✓ atomic epochs
- ✓ public, passive audit



What This Excludes

- ✗ cloud-side authoring or editing
- ✗ silent platform changes
- ✗ unverified first renders
- ✗ hidden publications



Compose



Sign
(orchestrator)



Hand over
(attested)



Ingest +
Countersign



Live +
Audit link



The publisher composes the value; the orchestrator signs it; the cloud carries it; anyone can check it.

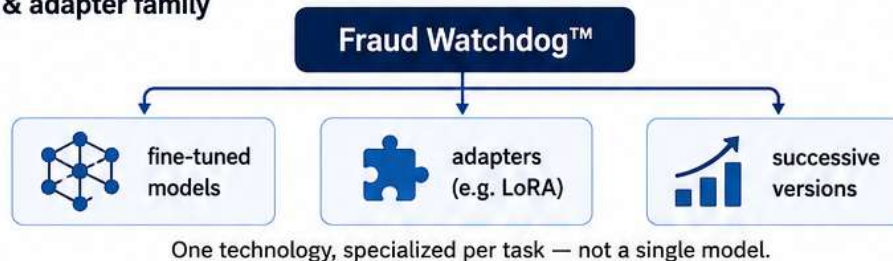


Fraud Watchdog™ — One Model Family, Two Watchposts

Scans on-prem and in the Workflow Ready Cloud (WRC)™. Flags and suspends — never edits.



1 A model & adapter family



2 Two watchposts, same technology

On-prem — Publisher Orchestrator



publisher-side role

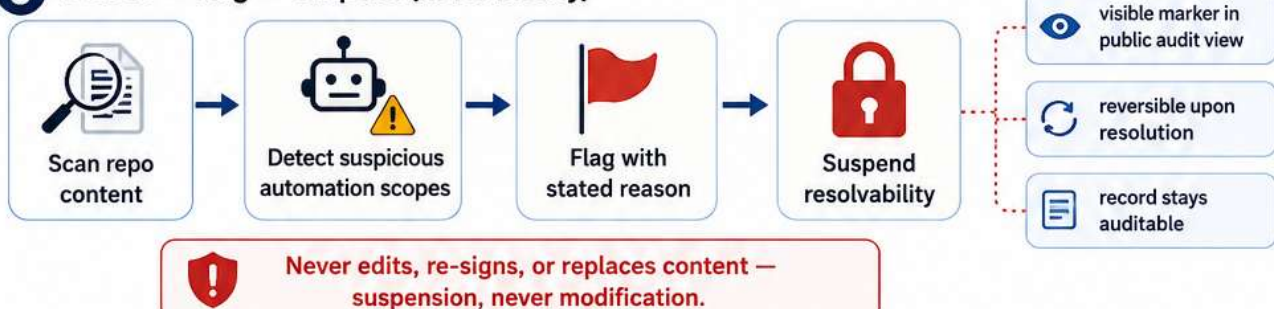


Workflow Ready Cloud (WRC)™



scans published public content, ongoing

3 Detect → Flag → Suspend (never modify)



4 One layer, not the judge



What This Guarantees

- ✓ ongoing scanning of public material
- ✓ reasoned, visible suspension
- ✓ auditable suspension record
- ✓ reversal path



What This Excludes

- ✗ silent content edits
- ✗ platform re-signing
- ✗ hidden takedowns
- ✗ scan result replacing consent



Fraud Watchdog™ can take content out of resolution — it can never change what anyone published.